RELATÓRIO TÉCNICO DE ARQUITETURA DE SOFTWARE

Sistema EmprestCar — Integração de API Gateway, REST, GraphQL e Mensageria RabbitMQ

Grupo 01: Artur Rabello, João Teixeira, Luis Fernando, Manassés Zenaro, Miguel Ângelo, Rafael Spaziani.
Disciplina: Engenharia de Software III | **Orientador:** Prof. EderRua

1. Visão Geral do Sistema e Arquitetura de Referência

O **EmprestCar** é uma plataforma peer-to-peer (P2P) de aluguel descentralizado de veículos que conecta diretamente proprietários (locadores) e condutores (locatários). A arquitetura do sistema foi projetada sob o padrão de **API Gateway na borda**, garantindo o isolamento completo da rede interna e a unificação das preocupações transversais (autenticação, controle de taxa, resiliência e correlação de logs).

Conforme a topologia de referência, o **API Gateway** escuta exclusivamente na porta pública **3000**. Atrás do Gateway, protegidos pela rede interna (escutando estritamente em 127.0.0.1), operam a **API REST Core** (porta 3001) e o servidor **GraphQL** (porta 4000). A comunicação assíncrona orientada a eventos utiliza o mensageiro **RabbitMQ 4** (portas 5672 AMQP e 15672 Painel Web em contêiner Docker).

TOPOLOGIA ARQUITETURAL DO SISTEMA EMPRESTCAR

```
[Clientes Externos: App Locatário / Painel Locador]
■ (Requisição HTTP + X-API-Key)
▼
[ API GATEWAY (Porta Pública 3000) ] ■■ Autenticação, Rate Limit, Circuit Breaker, Trace (X-Request-Id)
■ ■
■■ (Reescrita /api/locacoes) ■■ (Reescrita /api/graphql)
▼ ▼
[ API REST Core (127.0.0.1:3001) ] ■■■■■■■ [ Servidor GraphQL (127.0.0.1:4000) ]
(Recursos /veiculos e /locacoes) (Resolvers consomem REST síncrono)
■ (Publica evento 'locacao.criada')
▼
[ RABBITMQ BROKER (127.0.0.1:5672) ] ■■ Exchange Topic 'frota.eventos'
■■■ Fila 'fila.notificacao' ■■■ Consumidor 1 (Notificação E-mail/SMS)
■■■ Fila 'fila.frota' ■■■■■■■■ Consumidor 2 (Gestão de Frota / Limpeza)
■■■ DLX 'locacao.dlx' ■■■■■■■■■ Fila DLQ 'locacao.dlq' (Mensagens Mortas)
```

2. Mapeamento entre Documentação Conceitual e Código dos Tutoriais

Os artefatos de engenharia produzidos durante o semestre (ASRs, Caderno EARS, C4 Nível 3 / GRASP e ADR-001) foram integralmente mapeados nas implementações práticas dos tutoriais adaptados ao EmprestCar:

A. Requisitos Arquiteturalmente Significativos (ASRs) no Código:

• **ASR 1 (Consistência de Reservas / Zero Overbooking):** Implementado na API REST (rest/server.js) através da validação síncrona de conflito de datas no repositório. Quando detectada sobreposição para o mesmo veiculoId, o sistema interrompe a transação e retorna **409 Conflict** com padrão **RFC 9457**.
• **ASR 2 (Segurança e Controle de Acesso):** Verificado tanto na camada de API Gateway (recusando chaves inválidas com **401 Unauthorized**) quanto no serviço de pedidos/locações (conferindo se o X-Cliente-Id possui autorização sobre o recurso).
• **ASR 3 (Segurança das Transações):** Garantido pelo isolamento do estado 'Pendente de Pagamento' e pela confirmação

assíncrona confiável via RabbitMQ.
• **ASR 4 (Desempenho de Pesquisa):** Atendido pela paginação obrigatória (máximo 50 itens) na rota GET /veiculos e pela camada GraphQL, que consolida a busca em uma única requisição HTTP.

B. Caderno EARS (Sintaxe de Requisitos sem Ambiguidade):

• *Ubíquo:* Todo registro de locação possui UUID imutável, valor diário e datas vinculadas.
• *Orientado a Evento:* Ao criar a reserva (POST /locacoes), a API dispara imediatamente o evento locacao.criada para a exchange do RabbitMQ.
• *Orientado a Estado:* Enquanto um veículo estiver em manutenção ou reservado, o sistema impede novos agendamentos.
• *Comportamento Indesejado:* Requisições com CNH vencida ou condutores menores de 25 anos para carros da categoria LUXO são recusadas com status **422 Unprocessable Content**.

C. Componentes C4 Nível 3 (Padrões GRASP) e ADR-001:

• **Controller (Porta HTTP):** LocacaoController recebe as requisições REST/GraphQL.
• **Service (Caso de Uso):** LocacaoService coordena regras de negócio e validações.
• **Repository (Persistência):** LocacaoRepository encapsula a gravação no banco.
• **ADR-001 (UUID Imutável):** Definido que a troca de placa ou renovação de CNH não altera os IDs internos do veículo ou usuário, preservando o histórico contratual.

3. Tabela de Decisões Tecnológicas e Justificativas Arquiteturais

A tabela a seguir sumariza o papel de cada tecnologia no EmprestCar, a alternativa considerada e os impactos de sua adoção:

Tecnologia	Papel no EmprestCar	Por que foi Escolhida?	O que Aconteceria sem Ela?
API REST	Serviço core síncrono para CRUD de veículos e locações.	Padronização HTTP, semântica RFC 9110, erros RFC 9457 e facilidade de cache.	Dificuldade de integração; ausência de contrato padronizado entre clientes.
RabbitMQ	Mensageria assíncrona orientada a eventos de locação.	Desacoplamento temporal, resiliência via DLQ, idempotência e absorção de picos.	APIs síncronas lentas; falhas em cascata se o envio de e-mail/SMS cair.
GraphQL	Camada de agregação de consultas para a interface móvel.	Eliminação de over/under-fetching; busca agregada em 1 única requisição.	Múltiplas chamadas HTTP (3+ round-trips) para montar o painel do locatário.
API Gateway	Ponto único de entrada público (porta 3000) e isolamento.	Centralização de segurança (X-API-Key), Rate Limit e Circuit Breaker.	Exposição de serviços internos; duplicação de filtros em cada microserviço.

4. Análise de Limitações Técnicas do Protótipo e Soluções para Produção

Apesar de funcional e aderente aos requisitos da disciplina, a arquitetura do protótipo apresenta três limitações técnicas decorrentes do ambiente de demonstração local. Abaixo, analisamos os riscos e definimos as soluções arquiteturais exigidas para um ambiente de produção real:

Limitação 1 — API Gateway como Ponto Único de Falha (SPOF) e Rate Limit em Memória:

• *Risco no Protótipo:* O Gateway executa como uma única instância Node.js na porta 3000 e mantém as janelas de limite de uso (Rate Limiting) em uma estrutura Map em memória. Caso a instância caia ou seja escalada horizontalmente, o sistema fica indisponível ou os limites de requisição por cliente são zerados/multiplicados.

• *Solução em Produção:* Implantar o API Gateway em um cluster de instâncias redundantes (ex: Traefik ou Kong) atrás de um balanceador de carga externo (Cloud Load Balancer). Os contadores de limite de uso e estados dos clientes devem ser centralizados em um banco em memória distribuído e compartilhado (**Redis Cluster**).

Limitação 2 — Escrita Dupla e Risco de Inconsistência Eventual (REST x RabbitMQ):

• *Risco no Protótipo:* A API REST grava a locação em memória/banco e publica o evento locacao.criada na exchange em duas etapas sequenciais não transacionais. Se a aplicação falhar exatamente entre a gravação e a publicação, a reserva existirá no banco sem que os serviços de notificação e frota tomem conhecimento.
• *Solução em Produção:* Adotar o padrão **Transactional Outbox**. A gravação da locação e a inserção da mensagem na tabela outbox ocorrem na mesma transação ACID do banco PostgreSQL. Um processo assíncrono separado (*Debezium / Outbox Relayer*) lê a tabela e garante a publicação no RabbitMQ.

Limitação 3 — Comunicação Interna e Cabeçalho X-Cliente-Id Sem Verificação:

• *Risco no Protótipo:* Os serviços REST e GraphQL confiam no cabeçalho X-Cliente-Id injetado pelo Gateway. Se a rede interna não for estritamente isolada, uma aplicação invasora poderia forjar esse cabeçalho e acessar dados de outros usuários.
• *Solução em Produção:* Implementar autenticação mútua baseada em certificados digitais (**mTLS**) entre o Gateway e os microsserviços internos ou exigir que o Gateway assine um token **JWT interno de curta duração**, garantindo a autenticidade da identidade repassada.

5. Rastreabilidade Ponta a Ponta via X-Request-Id (Cenário Demonstrado)

Para comprovar a observabilidade e a rastreabilidade distribuída da arquitetura, executamos o cenário completo de criação de reserva para a cliente Mariana (estudante). A requisição gerou o identificador único req-uuid-99482-emp na borda e foi propagado através de todos os componentes:

FLUXO DE LOGS DO CENÁRIO DE LOCAÇÃO — X-Request-Id: req-uuid-99482-emp

```
1. API GATEWAY (Porta 3000):
[GATEWAY] req-uuid-99482-emp POST /api/locacoes -> Autenticado (cliente: app-locatario)
[GATEWAY] req-uuid-99482-emp Encaminhando para 127.0.0.1:3001/locacoes (RateLimit OK)

2. API REST CORE (Porta 3001):
[REST] req-uuid-99482-emp Recebido POST /locacoes. Validações EARS: CNH OK, Idade OK, Agenda OK.
[REST] req-uuid-99482-emp Reserva criada com sucesso (ID: 42). Resposta 201 Created enviada.
[REST] req-uuid-99482-emp Evento 'locacao.criada' publicado na exchange 'frota.eventos'.

3. MENSAGERIA RABBITMQ (Consumidores):
[NOTIFICAÇÃO] req-uuid-99482-emp Consumido evento da fila 'fila.notificacao'. Disparado E-mail/SMS.
[FROTA] req-uuid-99482-emp Consumido evento da fila 'fila.frota'. Painel do locador atualizado.

4. CAMADA GRAPHQL (Porta 4000):
[GRAPHQL] req-uuid-99482-emp Query 'PainelDoLocatario' executada. Resolvers consomem REST repassando
req-uuid-99482-emp.
```

6. Divisão do Trabalho e Declaração do Uso de Ferramentas de IA

A construção do trabalho prático envolveu todos os integrantes do Grupo 01, com a distribuição formal de responsabilidades detalhada na Tabela da Seção 1 deste relatório e do arquivo 00-indice.pdf.

Declaração de Uso de Ferramentas de IA: O grupo utilizou ferramentas de Inteligência Artificial Generativa (modelo Gemini / NotebookLM) como instrumento de apoio técnico para revisão de código, formatação dos modelos de

documentação (RFC 9457 e OpenAPI 3.2), geração de scripts de teste automatizados e diagramação visual dos relatórios em PDF. Todos os conceitos, regras de negócio do EmprestCar e decisões arquiteturais foram validados e corrigidos pela equipe.

7. Referências Bibliográficas

• FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. **RFC 9110: HTTP Semantics**. IETF, 2022.

• NOTTINGHAM, M.; WILDE, E.; DALAL, S. **RFC 9457: Problem Details for HTTP APIs**. IETF, 2023.

• HOHPE, G.; WOOLF, B. **Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions**. Boston: Addison-Wesley, 2003.

• RICHARDSON, C. **Microservices Patterns: With Examples in Java**. Shelter Island: Manning, 2018.

• NYGARD, M. T. **Release It! Design and Deploy Production-Ready Software**. 2. ed. Raleigh: Pragmatic Bookshelf, 2018.

• MAVIN, A. et al. **Easy Approach to Requirements Syntax (EARS)**. In: IEEE International Requirements Engineering Conference (RE), 2009.

• BROWN, Simon. **The C4 model for visualising software architecture**. 2026.